

Lab2 Demo

Yicheng CUI

2 Oct 2024

In this lab...

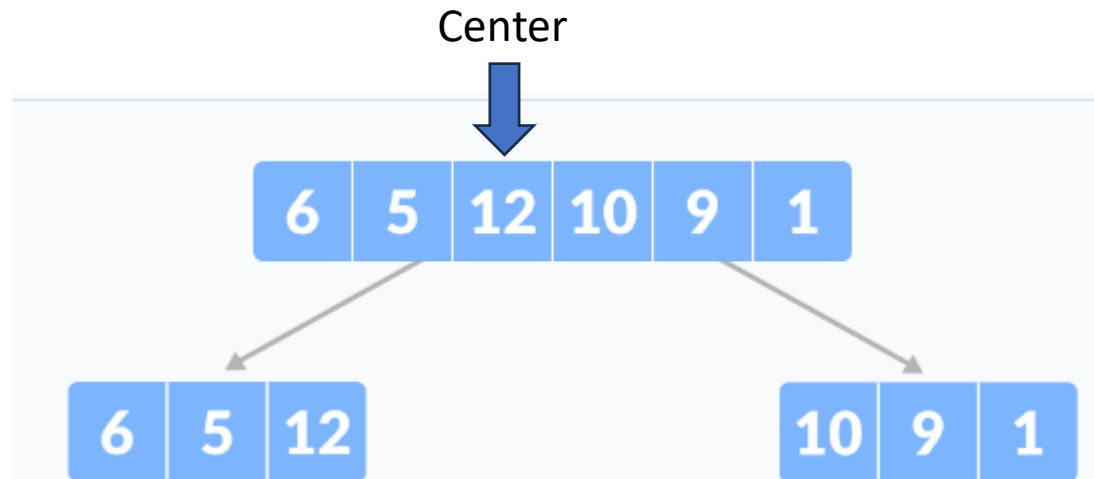
- You will be implementing different types of sorting.
- 1. Merge Sort (Demo)
- 2. Insertion Sort
- 3. Quick Sort
- 4. Generic Data Type Sorting (Bonus)

Merge Sort

- Q: Why do we need merge sort?
- A: It provides a fast and stable sorting solution. $O(n \log n)$
- Q: What is the fundamental operation in merge sort?
- A: Divide-and-conquer.

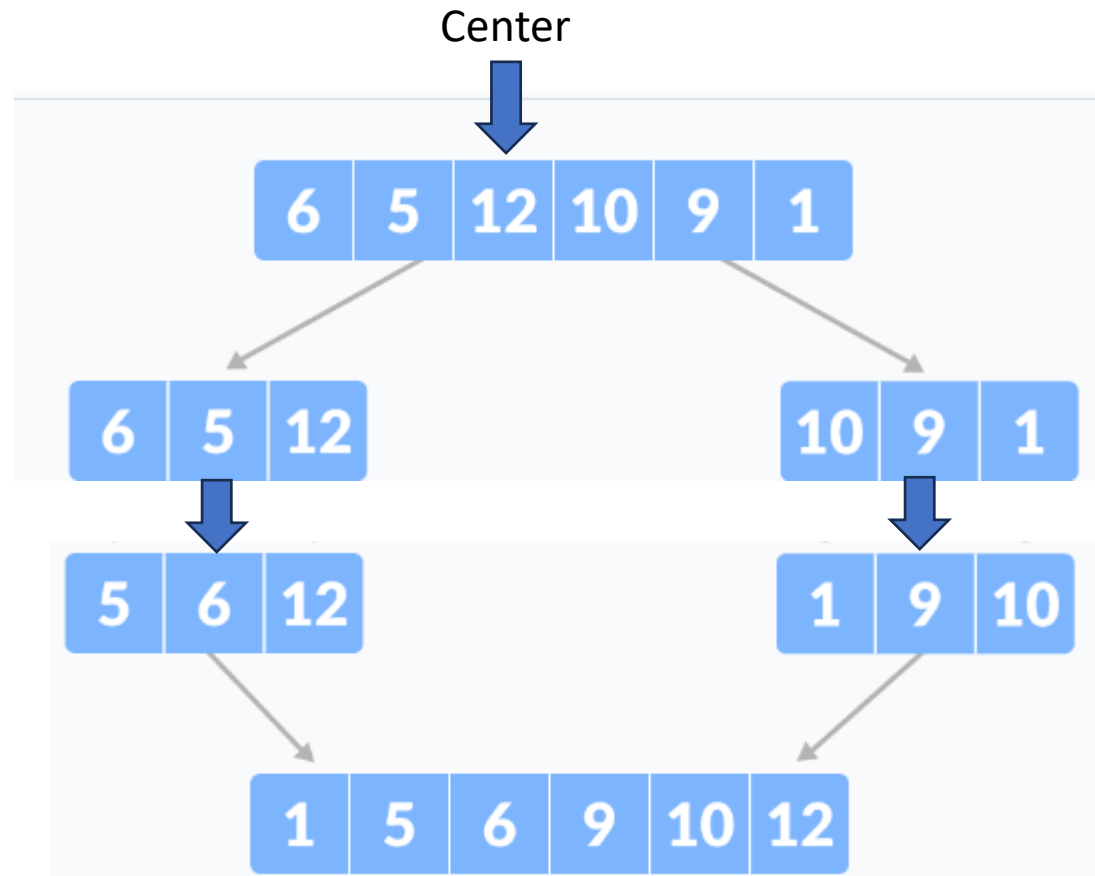
Merge Sort

- Divide: Split the array into two halves and solve the sorting problem independently.



Merge Sort

- Conquer: Merge the sorted sub-array to get back a whole sorted array.



Q1

Question 1: Merge Sort (40%)

You are asked to write a program to perform merge sort on an unsorted array of integers in descending order. If there are odd numbers in a certain iteration, one more number is divided to the left hand side of the array. Output the whole array after each conquer. A valid conquer refers to at least 2 numbers in the array. You can safely assume that there are no duplicate numbers in the array.

Your program should read the input from the file, and output the answer to another file. The first argument is the input file name, while the second argument is the output file name. Name your program as "lab2-q1.c".

Input file: First line, n, an integer describes the length of the array. Second line, n distinct integers, the unsorted array.

Output file: k lines, k is the number of conquer. Each line should have exactly n numbers separated by a space, indicating the result of the conquer in merge sort.

Q1: functions you need to modify

```
void merge(int a[], int tmpa[], int l, int r, int rend, int maxlen, FILE *fout)
```

Merge subarrays into one array

```
void msort(int a[], int tmpa[], int left, int right, int maxlen, FILE *fout)
```

Split the array to subarrays

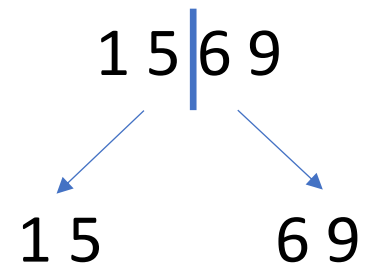
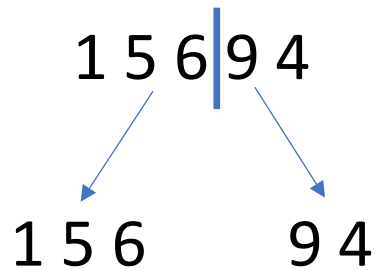
```
void mymergesort(int n, int a[], FILE *fout)
```

Call msort to apply merge sort.

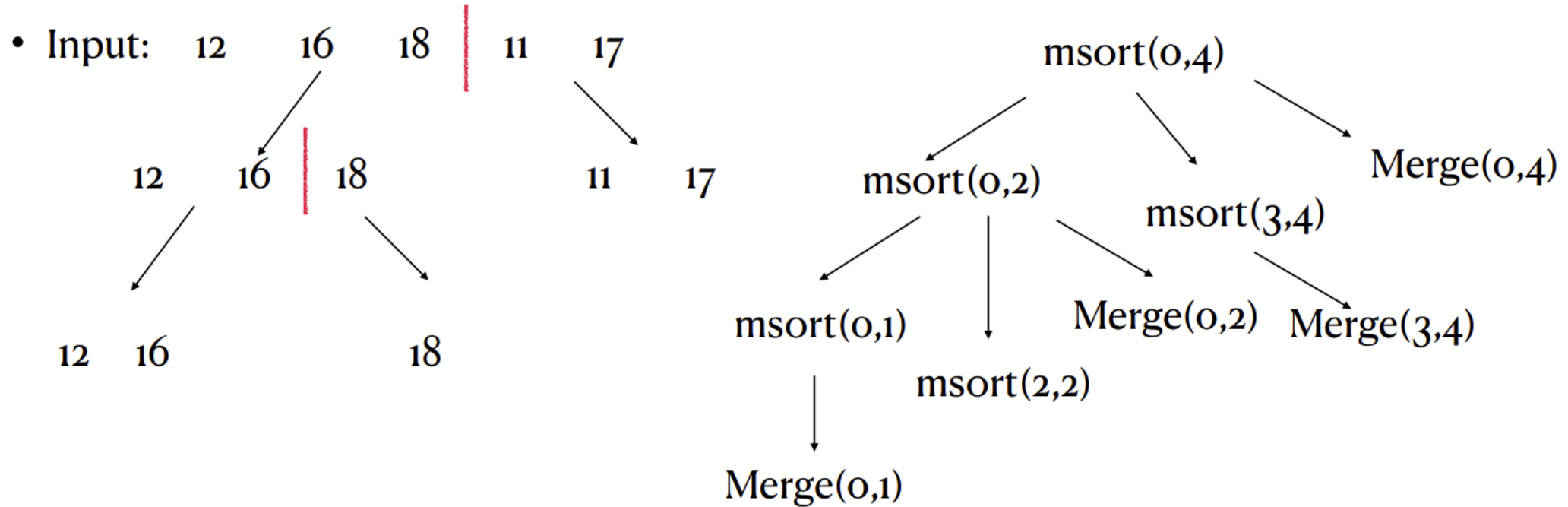
Q1: spilt arrays

```
void msort(int a[], int tmpa[], int left, int right, int maxlen, FILE *fout){  
    int center;  
    if (left < right){  
        center = (left + right) / 2;  
        msort(a, tmpa, left, center, maxlen, fout);  
        msort(a, tmpa, center + 1, right, maxlen, fout);  
        merge(a, tmpa, left, center + 1, right, maxlen, fout);  
    }  
}
```

If the number of elements in the array is odd, one more element is divided to the left hand side of the array.



Q1: spilt arrays



- Assume `msort(begin, end)` is the function for splitting the array to subarrays.

Q1: merge arrays

```
void merge(int a[], int tmpa[], int l, int r, int rend, int maxlen, FILE *fout){
    int i, t = l, lend = r - 1, n = rend - l + 1;
    if (n <= 1) return;
    while (l <= lend && r <= rend) /* main loop */
        if (a[l] >= a[r]) tmpa[t++] = a[l++];
        else tmpa[t++] = a[r++];
    while (l <= lend) /* copy rest of first half */
        tmpa[t++] = a[l++];
    while (r <= rend) /* copy rest of second half */
        tmpa[t++] = a[r++];
    for (i = 0; i < n; i++, rend--) /* copy tmpa back */
        a[rend] = tmpa[rend];
    for (i = 0; i < maxlen-1; i++)
        fprintf(fout, "%d ", a[i]);
    fprintf(fout, "%d\n", a[maxlen-1]);
}
```

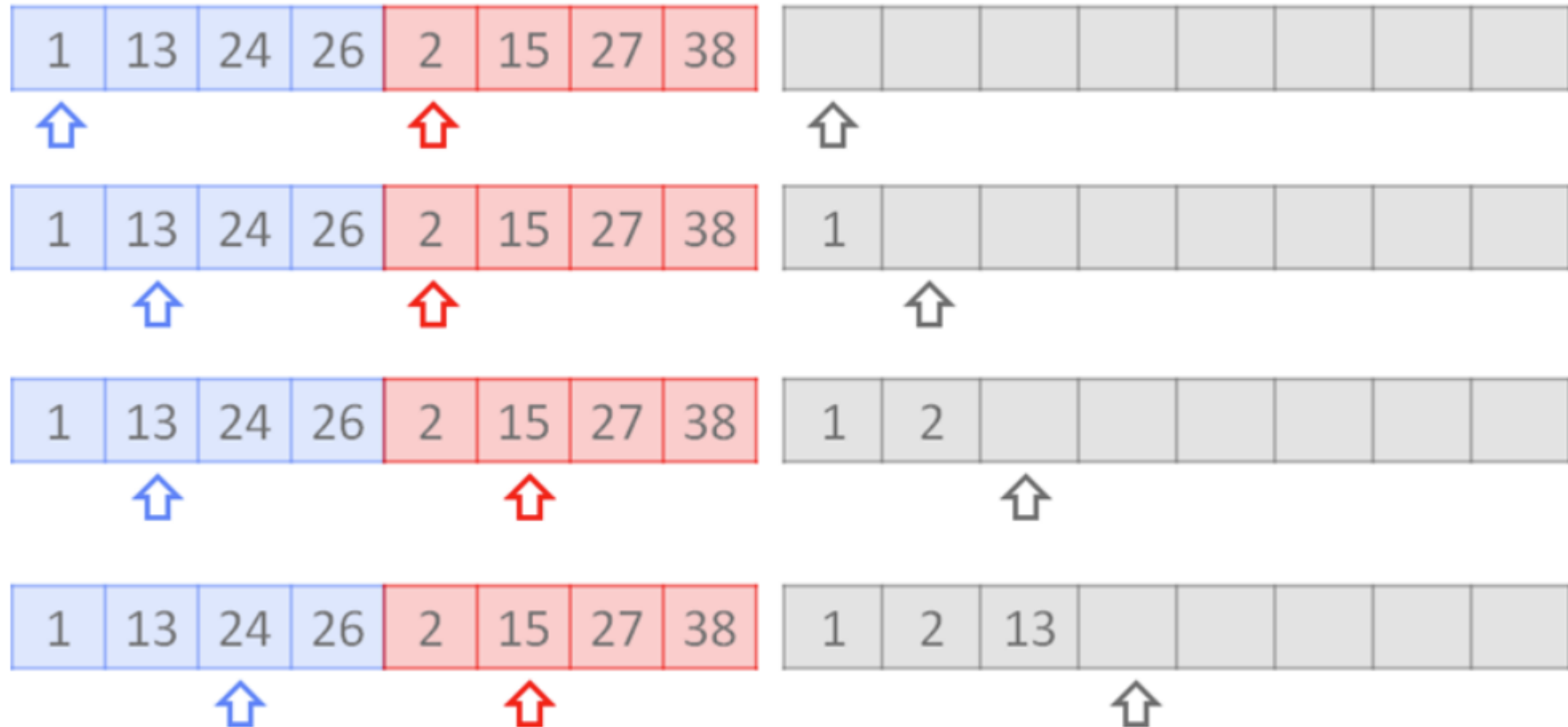
Sort the two subarrays

Copy the value from
two subarrays to one
array.

Note: do not print the result for merging one element. A valid conquer refers to at least 2 numbers in the array.

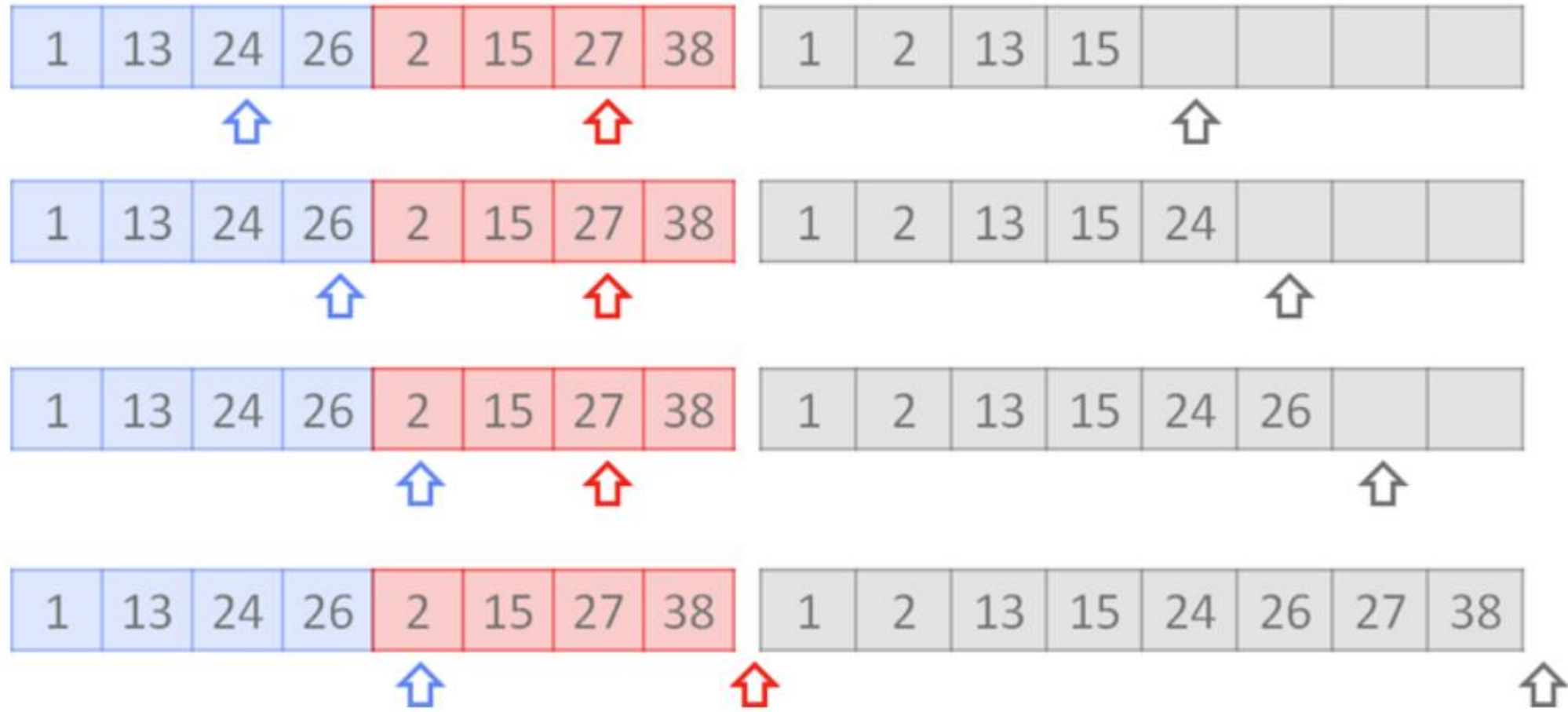
Q1: example

In Q1, we need to sort the array in descending order.



Q1: example (cont.)

In Q1, we need to sort the array in descending order.



Q1: mymergesort

```
void mymergesort(int n, int a[], FILE *fout){    /* driver */  
    int *tmpa = (int *)malloc(n * sizeof(int)); /* memory for merged list */  
    msort(a, tmpa, 0, n - 1, n, fout);  
    free(tmpa);  
}
```

1. allocate the memory to the tmpa array
2. Call the msort method
3. Free the array tmpa

Hints for Q2 and Q3

- For insertion sort and quick sort, you can follow the style and genre of what we have done in Q1.
- In addition, you can work on our starting code. You can use the codes given in lecture slide 4. However, you need to modify them according to the requirement of the questions.

Enjoy your lab!

Thanks!